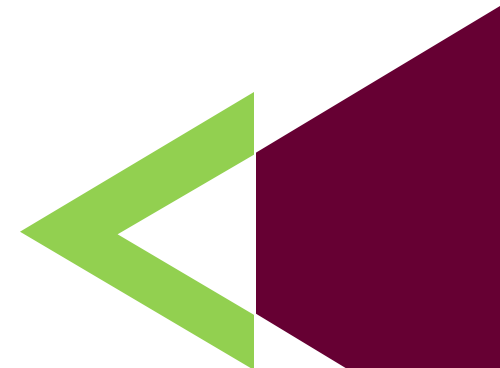


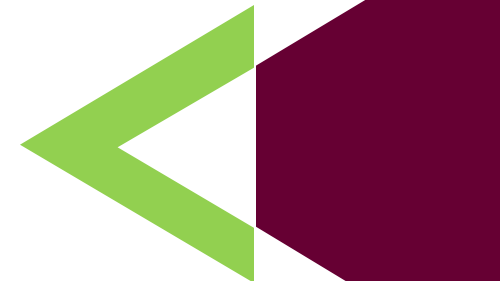


TEMA 2: DWECC (Desarrollo Web Entorno Cliente)



1 Fundamentos de JavaScript I

- Cuando se empezó a usar JavaScript Microsoft sacó su versión llamada JScript y hubo problemas de compatibilidades. Por ello el W3C sacó un estándar para el modelo de objetos del documento de una página (DOM) y el estándar ECMA que recoge las especificaciones de JavaScript desde 1997.
- Es similar en sintaxis a C y tiene algunos nombres y convenciones como Java (Aunque no tiene nada que ver con Java).
- Su creador fue Brendan Eich que trabajaba en Netscape. En la actualidad es una marca registrada por Oracle.



1 Fundamentos de JavaScript II (comentarios)

- Los comentarios son sentencias que el intérprete de JavaScript ignora.
- En el caso de JavaScript los comentarios en el código ocupan sitio y deben ser descargados con el resto del código por el navegador (por un lado, es recomendable documentar lo más posible el código, pero por otro lado implica más descarga por los navegadores).



1 Fundamentos de JavaScript III (comentarios)

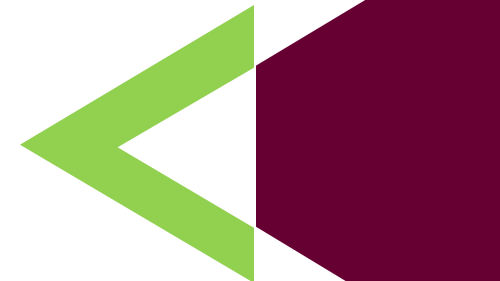
- Como casi todos los lenguajes de programación tenemos dos tipos de comentarios:

- De una única línea: Se realizan con dos barras inclinadas //

```
// Este es un comentario de una línea
```

- De múltiples líneas: se inician con /* y se cierra con */

```
/* Ésta es una sección  
de comentarios  
en el código de JavaScript */
```



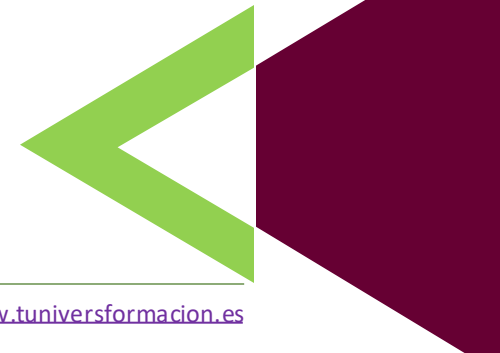
1 Fundamentos de JavaScript IV (variables)

- Las variables en JavaScript se definen con la palabra reservada var (se puede dar valor en la propia definición o a posteriori:

```
var edad = 38;
```

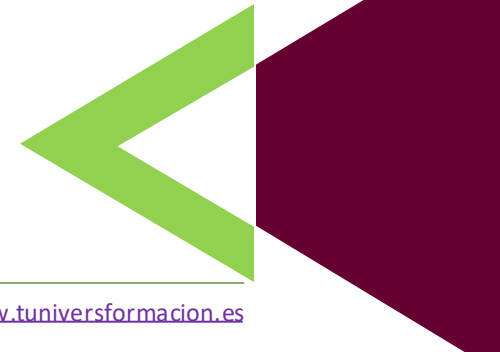
```
var edad;  
edad = 38;
```

- En JavaScript las variables pueden almacenar cualquier tipo de datos, no indicamos de que tipo son (Lenguaje no tipado o de tipado dinámico)



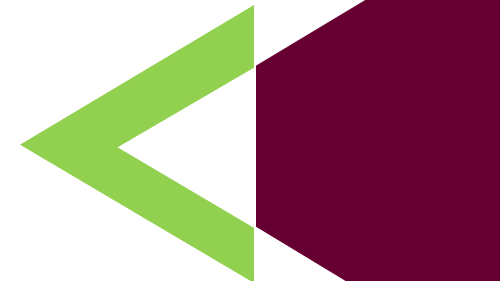
1 Fundamentos de JavaScript V (variables)

- Como en casi todos los lenguajes de programación los nombres de las variables aceptan caracteres alfanuméricos y el carácter de subrayado.
- No se pueden utilizar otros caracteres como el espacio, símbolos (+, -, %, ...)
- No se podrán utilizar palabras reservadas del lenguaje
- Normalmente suele ser recomendable utilizar camelcase para los nombres.



1 Fundamentos de JavaScript VI (tipos de datos)

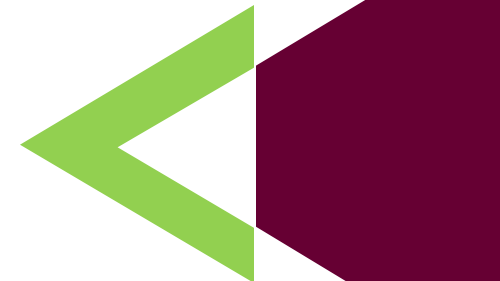
- String: Se crean con un texto entre comillas. Dentro se puede usar:
 - Salto de línea \n
 - Comillas dobles \"
 - Tabuladores \t
- Number: No diferencia de enteros y flotantes. Permite además de la notación en base 10 común:
 - Notación científica: 2.6e-5
 - Base 8: 0234
 - Base 16: 0X2AFE



1 Fundamentos de JavaScript VII (tipos de datos)

- Boolean: Valores true o false.
- Object: Cualquier objeto definido por el usuario y objetos del lenguaje (document, array, etc.)
- Null: valor null
- Function: Definición de una función

```
var miArray = (2,5,7);  
//objetos  
var miObjeto = {  
  propiedad: 23,  
  otracosa: "hola"  
}
```



1 Fundamentos de JavaScript VIII (Conversiones)

- Al intentar sumar un número con una cadena (que sea también un número) convierte automáticamente el número a cadena y los concatena.

```
4 + 5 + "6" // resultado = "96"
```

- Se pueden convertir cadenas a números con las funciones:

- parseInt()

- parseFloat()

```
parseInt("34")
```

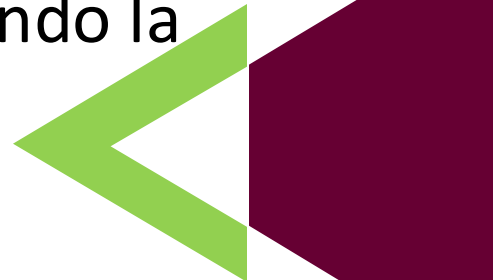
```
parseInt("89.76")
```

```
parseFloat("34")
```

```
parseFloat("89.76")
```

- Se pueden convertir números a cadenas simplemente sumando la cadena vacía delante.

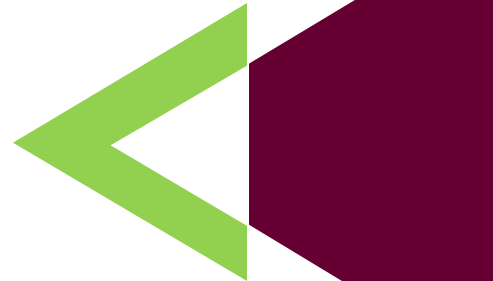
```
("" + 3400)
```



1 Fundamentos de JavaScript IX (Operadores)

- Operadores de comparación

- > Mayor que
- < Menor que
- >= Mayor o igual que
- <= Menor o igual que
- == Igual que (JavaScript transforma al mismo tipo de dato '1' == 1 es true)
- === Igual estricto ('1' === 1 es false)
- != Diferente
- !== Diferente estricto



1 Fundamentos de JavaScript X (Operadores)

- Operadores aritméticos

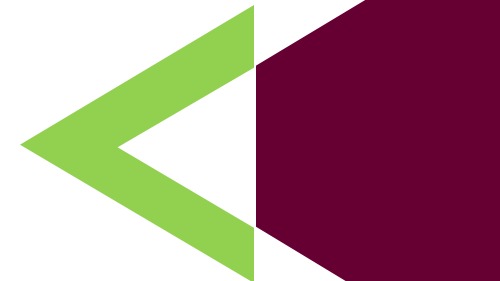
- + Sumar dos números o concatenar dos cadenas
- - Restar
- * Multiplicar
- ** Exponente
- / Dividir
- % Módulo
- ++ Incremento (operador unario variable++)
- -- Decremento (operador unario variable--)
- - Valor negativo (operador unario -variable)
- + Valor convierte a número una cadena como la función Number() (Number(es diferente a parseFloat()) -> BUSCAR



1 Fundamentos de JavaScript XI (Operadores)

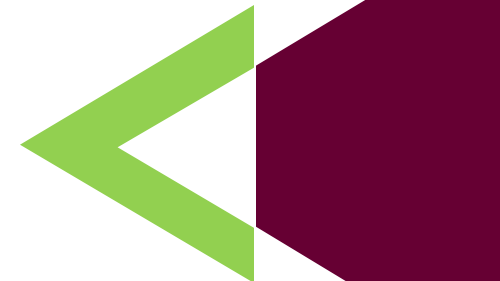
- Diferencias entre Number() y parseFloat()
 - parseFloat() Es una función implementada en JavaScript
 - Number() Es el constructor de la clase
 - parseFloat() solamente puede transformar a números de base 10
 - parseFloat() Si encuentra un carácter diferente a un número, un punto, un símbolo +/- o un exponente deja de convertir en ese punto y devuelve el número hasta ese carácter.
 - Number() Si la cadena no puede ser convertida devuelve NaN

¿Cómo funcionarían con 3.14PI y con 0x20?



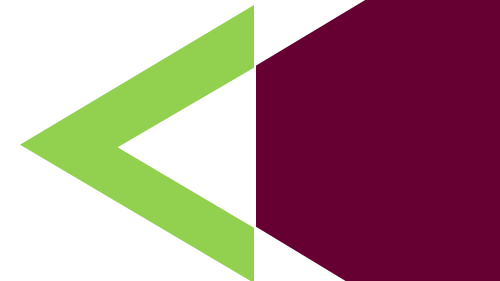
1 Fundamentos de JavaScript XII (Operadores)

- Operadores de asignación
 - = Asignación simple
 - += Sumar y asignar el valor
 - -= Restar y asignar un valor
 - *= Multiplicar y asignar un valor
 - **= Exponenciación y asignar un valor
 - /= Dividir y asignar un valor
 - %= Calcular módulo y asignar el valor



1 Fundamentos de JavaScript XIII (Operadores)

- Operadores de asignación (bitwise). Trabajan con 32 bits aunque el número se almacena con 64 bits
 - $\ll=$ Desplazamiento de bits a la izquierda y asignar. Se rellena con 0 por la derecha.
 $a=5 \mid a\ll=1 \rightarrow a=5\ll 1 \mid 101 \rightarrow 1010 \mid a=10$
 - $\gg=$ Desplazamiento de bits a la derecha y asignar. Se denomina con signo, rellena con 0 o 1 por la izquierda según el número sea positivo o negativo.
 $a=5 \mid a\gg=1 \rightarrow a=5\gg 1 \mid 101 \rightarrow 010 \mid a=2$



1 Fundamentos de JavaScript XIV (Operadores)

- Aclaración números binarios positivos y negativos (en complemento 2). Para convertir un número positivo en binario a negativo primero se obtiene el número inverso bit a bit y después se suma 1. De esta forma siempre el número empezará con un 1 a la izquierda.

➤ 40

➤ 00101000

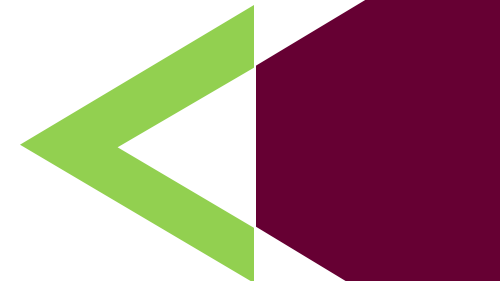
➤ 11010111

➤ $11010111 + 1 \rightarrow 11011000 \rightarrow -40$



1 Fundamentos de JavaScript XV (Operadores)

- Operadores de asignación (bitwise).
 - `>>>=` Se desplazan los bits a la derecha y se rellena con 0 por la izquierda.
- Los usos de estas operaciones son:
 - Multiplicación o división por potencias de 2.
Normalmente es más eficiente que ejecutar la multiplicación -> optimización
 - Operación con bits con o sin signo

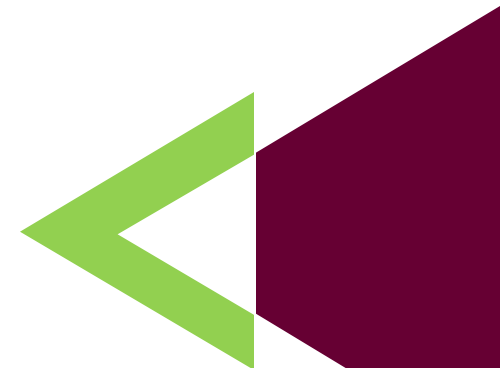


1 Fundamentos de JavaScript XVI (Operadores)

- Operadores de asignación (lógicas bit a bit).

- $\&=$ Operación AND bit a bit y asignación
- $|=$ Operación OR bit a bit y asignación
- $\wedge=$ Operación XOR bit a bit y asignación

ENTRADAS						
A	B	AND	NAND	OR	NOR	EXOR
0	0	0	1	0	1	0
0	1	0	1	1	0	1
1	0	0	1	1	0	1
1	1	1	0	1	0	0
funciones		$A \cdot B$	$A \cdot \bar{B}$	$A + B$	$\bar{A} + \bar{B}$	$A \oplus B$



1 Fundamentos de JavaScript XVII (Operadores)

- Operadores de asignación (desestructuración).
 - Permite sacar de un objeto sus propiedades o elementos de un array y asignarlos a unas variables.
 - De forma básica podría usarse de la siguiente manera

```
[a,b]=[c,d]  a=c, b=d
```



1 Fundamentos de JavaScript XVIII (Operadores)

- Operadores de asignación (desestructuración).

➤ En el caso de objetos funcionaría así:

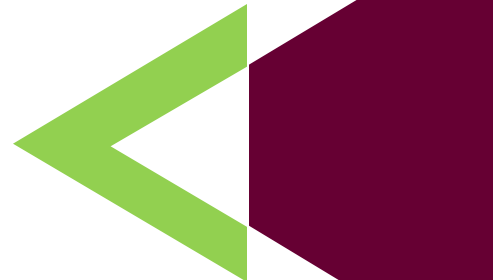
```
// Create an Object
```

```
const person = {  
  firstName: "John",  
  lastName: "Doe",  
  age: 50  
};
```

```
// Destructuring
```

```
let {firstName, lastName} = person;
```

- Notar que en este caso el operador usa {} en vez de []
- En el caso de objetos comprobará si alguna propiedad se llama como la variable a la que estamos haciendo la asignación
- Podemos dar valores por defecto por si no existen:
 - Let {firstName,country="US"} =person



1 Fundamentos de JavaScript XIX (Operadores)

- Operadores de asignación (desestructuración).

- En el caso de cadenas funcionaría así:

```
let name = "Testing";
```

```
// Destructuring
```

```
let [a1, a2, a3, a4, a5] = name;
```

- En este caso usamos []
- En cada una de las variables irá introduciendo las letras de la cadena empezando por la izquierda y en orden consecutivo.



1 Fundamentos de JavaScript XX (Operadores)

- Operadores de asignación (desestructuración).

- En el caso de arrays

```
// Create an Array
const fruits = ["Bananas", "Oranges", "Apples", "Mangos"];

// Destructuring
let [fruit1, fruit2] = fruits;
```

- En este caso usamos []
- Ira asignando a cada variable un elemento del array.
- Podemos saltarnos valores con comas [fruit1,,fruit2]
- Podemos utilizar ...rest para el resto del array [fruit1,fruit2, ...rest]
- Podemos seleccionar posiciones [[1]:fruit1,[2]:fruit2]



1 Fundamentos de JavaScript XXI (Operadores)

- Operadores de asignación (desestructuración).
 - Se puede utilizar en más ocasiones:
 - Maps
 - Tomando los valores que devuelve una función
 - En bucles for of



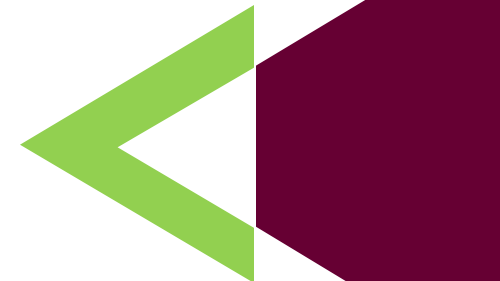
1 Fundamentos de JavaScript XXII (Operadores)

- Operadores booleanos

- && AND

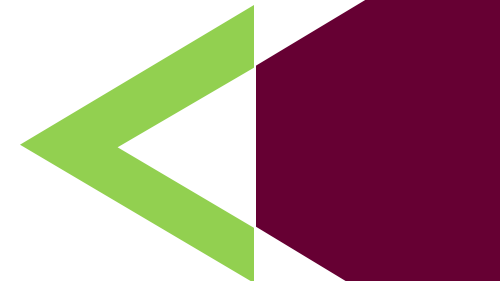
- || OR

- ! NOT



1 Fundamentos de JavaScript XXIII (Operadores)

- Operadores bit a bit.
 - & AND bit a bit
 - | OR bit a bit
 - ^ XOR bit a bit
 - ~ NOT bit a bit
 - << Desplazamiento a la izquierda
 - >> Desplazamiento a la derecha con signo
 - >>> Desplazamiento a la derecha sin signo



1 Fundamentos de JavaScript XXIV (Operadores)

- Operadores de objeto.
 - . Operados para acceder a una propiedad o método de un objeto
 - [] Para enumerar miembros de un Array
 - delete (delete oceanos[2]) para borrar elementos de un array (queda el elemento como undefined)
 - in para saber si existe una propiedad o un método en el objeto ("write" in document)
 - instanceof para comprobar si un objeto es una instancia de un objeto nativo de JavaScript (a instanceof Array)



1 Fundamentos de JavaScript XXIV (Operadores)

- Operadores misceláneos

- Operador condicional ternario ?:

- Condición ? Expresión si se cumple : expresión si no se cumple

```
var a,b;  
a = 3; b = 5;  
var h = a > b ? a : b;
```

- Typeof permite identificar si una variable pertenece a number, string, boolean, object, function o undefined
 - typeof miVariable



1 Fundamentos de JavaScript XXV (Condiciones y bucles)

- Estructuras de control

- if

```
if (condición)      // entre paréntesis irá la condición que se evaluará a true o false.
{
    // instrucciones a ejecutar si se cumple la condición
}
```

- if ...else

```
if (condición)      // entre paréntesis irá la condición que se evaluará a true o false.
{
    // instrucciones a ejecutar si se cumple la condición
}
else
{
    // instrucciones a ejecutar si no se cumple la condición
}
```



1 Fundamentos de JavaScript XXVI (Condiciones y bucles)

- Estructuras de control

- If...else if

```
if (condicion){ //condicion entre parentesis
    //codigo a ejecutar si se cumple la condicion
}
else if (condicion2){ //segunda condicion entre parentesis
    //codigo a ejecutar si no se cumple la primera pero si la segunda
}
else{
    //codigo a ejecutar si no se cumple ninguna
}
```

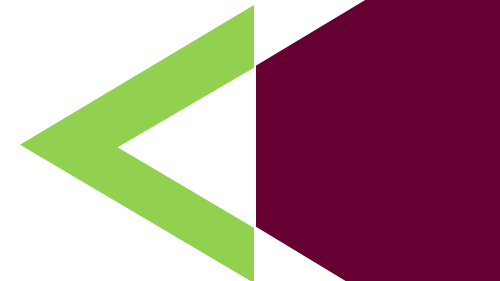


1 Fundamentos de JavaScript XXVII (Condiciones y bucles)

- Estructuras de control

- switch

```
switch (variable){  
  case valor:  
    //Código a ejecutar si la variable tiene el valor  
    break; //Si se cumplio no queremos seguir comprobando otros casos  
  case otrovalor:  
    //Código a ejecutar si la variable tiene el otro valor  
    break;  
  //.....  
  default:  
    //Código a ejecutar si no se cumplio ningún otro caso  
}
```

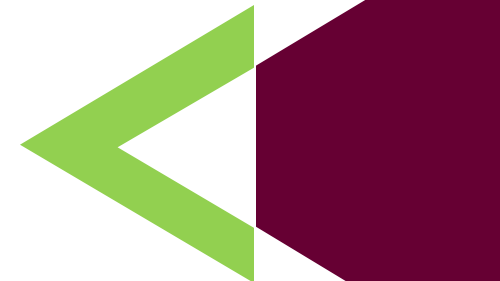


1 Fundamentos de JavaScript XXIX (Condiciones y bucles)

- Bucle

- for (expresión inicial; condición; incremento). Sirve para repetir un número limitado de veces unas instrucciones

```
for (var i=1; i<=20; i++)  
{  
    // instrucciones que se ejecutarán 20 veces.  
}
```



1 Fundamentos de JavaScript XXX (Condiciones y bucles)

- **Bucles**

- **while (condición).** Repetir un número indefinido de veces siempre que se cumpla una condición.

```
while (condición)
{
    // Instrucciones a ejecutar dentro del bucle.
}
```

- Cuidado con asegurarse que la condición se cumplirá en algún momento



1 Fundamentos de JavaScript XXXI (Condiciones y bucles)

- Bucle

- Do while (condición). Como en el caso anterior, pero nos aseguramos de que las instrucciones se ejecutan al menos una vez.

```
do {  
    // Instrucciones a ejecutar dentro del bucle.  
}while (condición);
```

